

```

import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, KFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load the breast cancer dataset
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

Imported all the necessary libraries and loaded the dataset as shown in the canvas.
Split the dataset into Training and testing sets and 20% of the data was used for testing.

```

# Initialize the Random Forest classifier
rf = RandomForestClassifier(random_state=42)

# Define the parameter grid for GridSearchCV
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Perform GridSearchCV to find the best parameters
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Get the best parameters
best_params = grid_search.best_params_
print("Best Parameters:", best_params)

```

Initialized a Random Forest classifier and the grid of hyperparameters to search over.
Performed an search over the specified parameter values for the estimator with GridSearchCV
Used 5-fold cross-validation , accuracy as a metric to evaluate the model. And best_params retrieves the best parameters found by GridSearchCV.

```

# Initialize the Random Forest classifier with the best parameters
best_rf = RandomForestClassifier(**best_params, random_state=42)
best_rf.fit(X_train, y_train)

```

Initialized a new Random Forest classifier with the best parameters found by GridSearchCV and fits the model on the training data.

```
# Perform k-fold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(best_rf, X_train, y_train, cv=kf, scoring='accuracy')

print("Cross-Validation Scores:", cv_scores)
print("Mean CV Accuracy:", np.mean(cv_scores))
```

Splits the dataset into k consecutive folds and evaluates a score by cross-validation. Used accuracy as the metric to evaluate the model.

```
# Make predictions on the test set
y_pred = best_rf.predict(X_test)

# Calculate evaluation metrics
test_accuracy = accuracy_score(y_test, y_pred)
test_precision = precision_score(y_test, y_pred)
test_recall = recall_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred)

# Print the results
print("\nTest Set Performance:")
print(f"Accuracy: {test_accuracy:.4f}")
print(f"Precision: {test_precision:.4f}")
print(f"Recall: {test_recall:.4f}")
print(f"F1 Score: {test_f1:.4f}")
```

Made predictions on the test set and calculated the respective metrics for the test set. Prints the performance metrics in the end.